CMPUT 333 Assignment 2

Group 15: Sam Francisco, Xinmeng Zheng, Miko Punsalan

Winternitz-Like One-Time Signature

Environment Setup and Running

- Environment requirements: Ubuntu 24.04 (using multipass) and Python 3.12.
- To make the programs executable, please grant access by command `chmod +x ots_genkeys.py ots_sign.py ots_verify.py`.
- To generate a key pair, run `ots_genkeys.py by python3 ots_genkeys.py`. This creates `private_key.ots` and `public_key.ots`.
- To sign a message, run `python 3 ots_sign.py message_files/message.txt private_key.ots`. This produces `signature.ots`.
- To verify a signature, run `python3 ots_verify.py message_files/message.txt public_key.ots signature.ots`. Output would be VALID or INVALID.

Validating and Testing

We performed several tests to check the correctness of our programs.

Test 1: Valid Signature

```shell
echo -n "hello" > message_files/m1.txt

python3 ots_genkeys.py

python3 ots_sign.py message_files/m1.txtt private_key.ots

python3 ots_verify.py message_files/m1.txt public_key.ots signature.ots
```

Expected result:

```shell
VALID
```

This confirms that a correctly generated signature verifies successfully.

Test 2: Modified/Corrupted Message

After signing the message, modify the file contents and try to verify again:

```shell
echo "extra" >> message_files/m1.txt

python3 ots_verify.py message_files/m1.txt public_key.ots signature.ots
```

Expected Result:

```shell
INVALID
```

This confirms that a signature is tied to the exact message contents.

Test 3: Modified/Corrupted Signature

Flip one byte in the signature file and verify it:

```shell
python3 - <<EOF

data = bytearray(open("signature.ots","rb").read())

data[0] ^= 1

open("signature_changed.ots","wb").write(data)

EOF

python3 ots_verify.py message_files/m1.txt public_key.ots signature_changed.ots
```

Expected output:

```shell
INVALID
```

This confirms that any change in the signature fails verification.

Test 4: Wrong Public Key

Generate a new key pair and verify the original signature:

```shell
python3 ots_genkeys.py

python3 ots_verify.py message_files/m1.txt public_key.ots signature_old.ots
```

Expected output:

```shell
INVALID
```

This confirms that a signature can only be verified using the correct public key.

Unique Key Generation

By using os.urandom(32), ots_genkeys generates 64 private 32-byte seeds independently for each key pair. Thus, the probability of generating the same key pair is very small.

Probability of Producing Valid Signature for Random File

In this simplified WOTS implementation, checksum is not used. For a given message hash, each nibble d_i (i = 0, 1, 2, …, 15) determines how many times the private seed is hashed to produce $H_{d_i}(seed_i)$. Since hash chains only move forward, a signature component can only be used to generate valid values for nibble values d'_i greater than or equal to the original nibble d_i, i.e. $d'_i \geq d_i$ for all nibbles. The probability of a random nibble satisfies $d'_i \geq d_i$ is around $1/2$, since each nibble of a SHA-256 hash is roughly uniformly distributed between 0 and 15. Thus, the probability that all nibbles satisfy $d'_i \geq d_i$ is $(1/2)^{64}$.

Therefore, if we have obtained the valid signature of this file, the probability we can produce a valid signature for another random file is roughly $(1/2)^{64}$.

Why Simple Checksum is Secure in WOTS

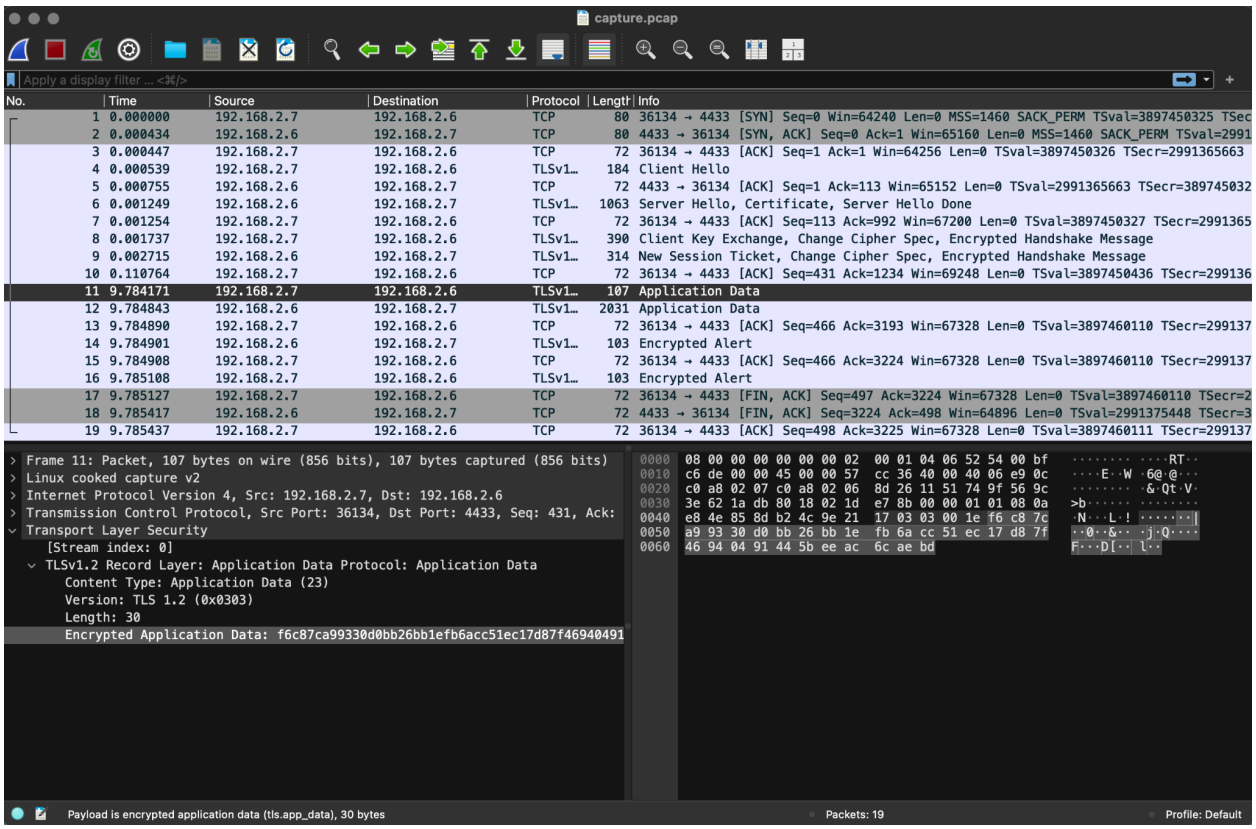
In WOTS, checksum is appended to the message hash and also signed. It ensures that the total number of hashing across all chains is fixed. If some nibble values are increased (move forward in the hash chain), the checksum would force other nibbles to decrease. Hash functions are

one-way, decreasing a nibble value (move backwards in the hash chain) is very infeasible. Thus, the checksum effectively protects signatures from being extended.

Demonstration of Perfect Forward Secrecy

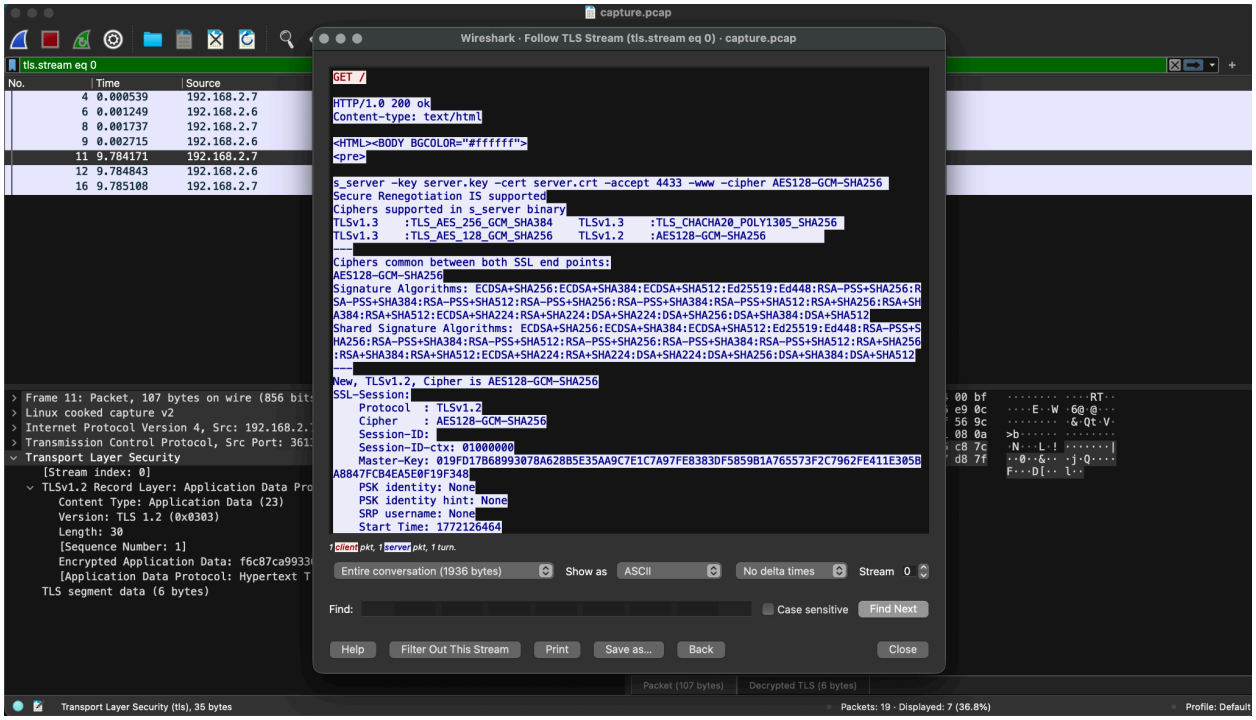
Scenario 1: No PFS

Note that servervm's IP is 192.168.2.6 and clientvm's IP is 192.168.2.7. After opening capture.pcap and before providing tls-keys.log to Wireshark, packet 11 shows:



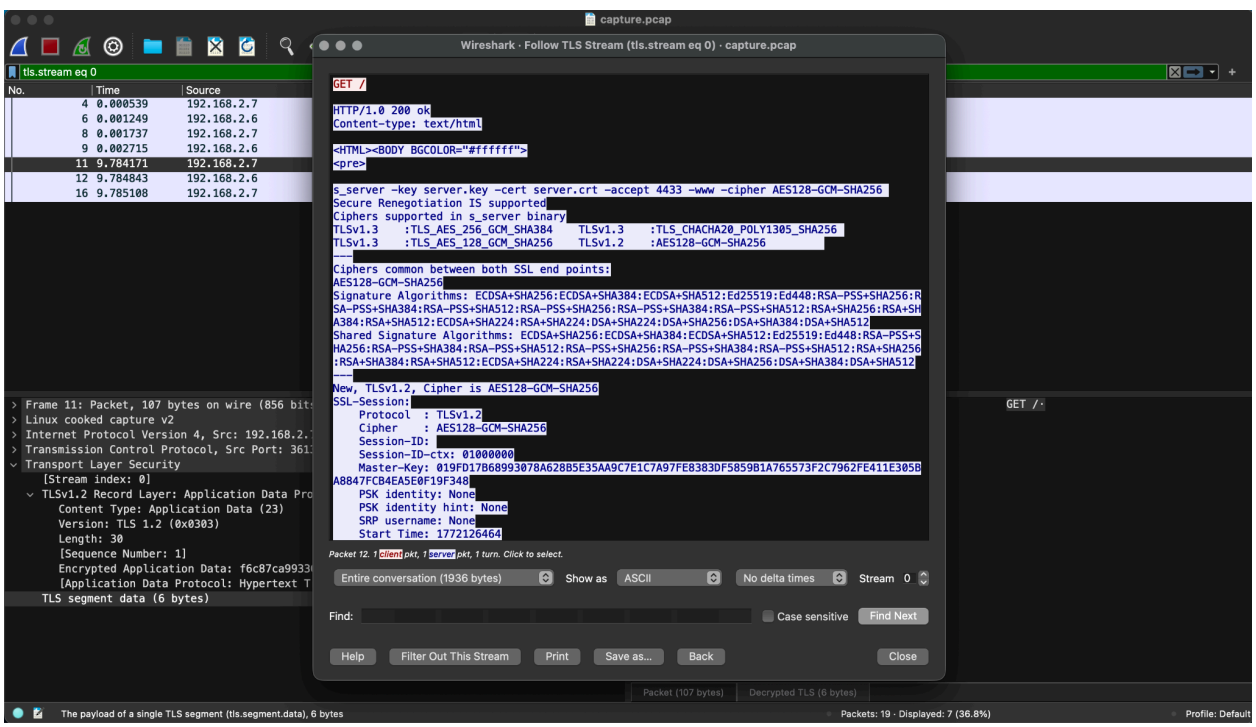
Case 1.1

After providing tls-keys.log, the TLS segment data sent from the client to the server can now be decrypted and is shown to contain "GET /.":



Case 1.2

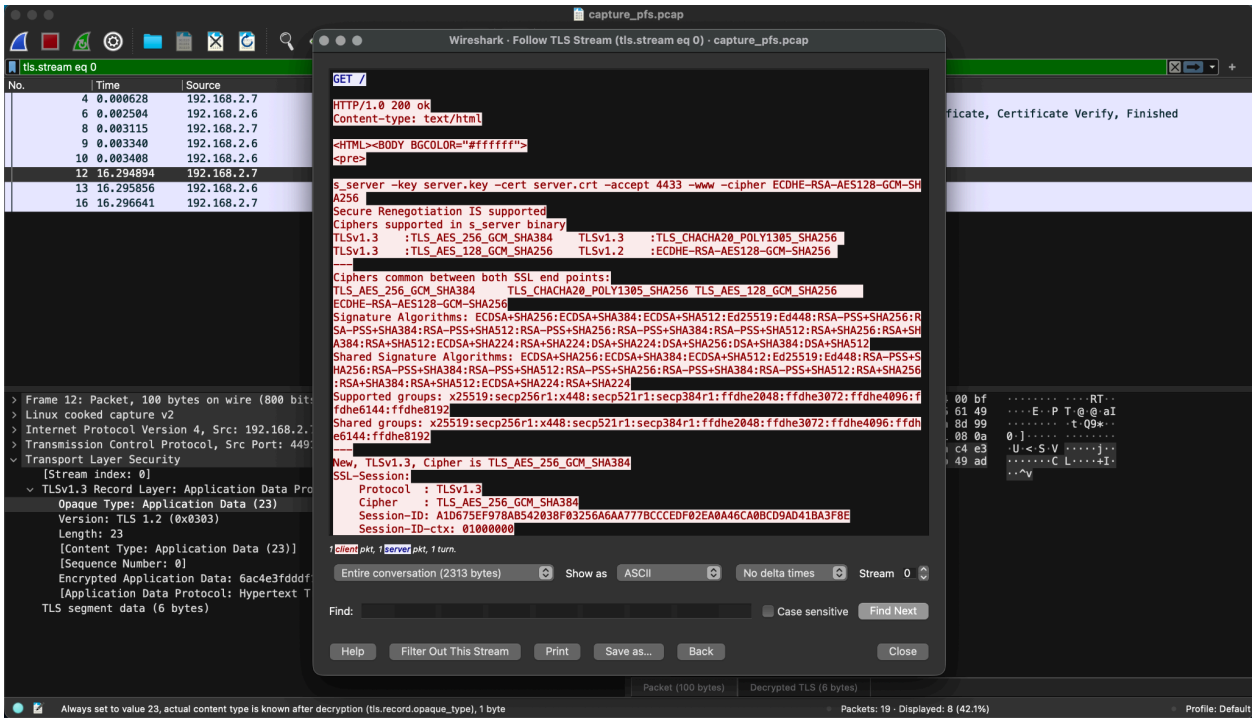
The same can be achieved when removing tls-keys.log and providing (`IP=192.168.2.6, Port=4433, Protocol=http, Key File=server.key`) to the RSA keys list instead:



Scenario 2: PFS

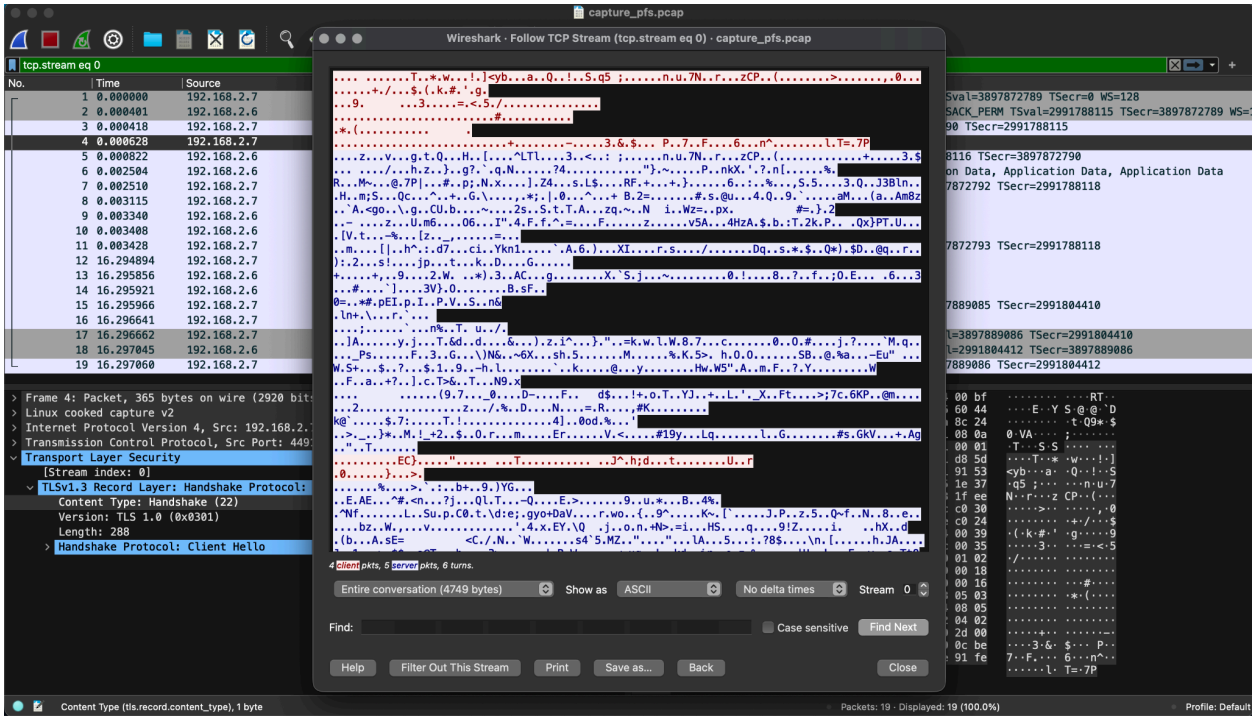
Case 2.1

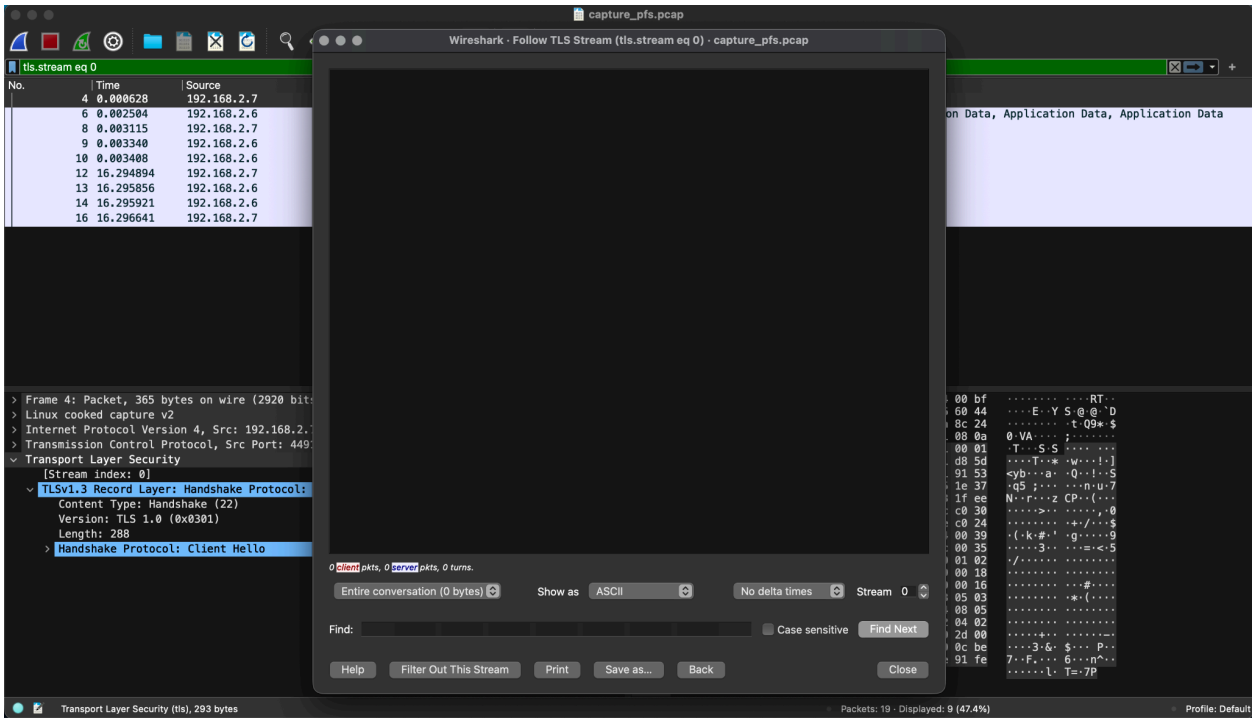
Opening capture_pfs.pcap and providing tls-keys-pfs.log, packets can be decrypted and exposes the client request to be packet 12 and the response in packet 13:



Case 2.2

After removing tls-keys-pfs.log and providing (`IP=192.168.2.6, Port=4433, Protocol=http, Key File=server.key`) to the RSA keys list, application data is unable to be decrypted:


capture_pfs.pcap
Wireshark · Follow TCP Stream (tcp.stream eq 0) · capture_pfs.pcap
tcp.stream eq 0
No. Time Source
1 0.000000 192.168.2.7
2 0.000401 192.168.2.6
3 0.000418 192.168.2.7
4 0.000628 192.168.2.7
5 0.000822 192.168.2.6
6 0.002504 192.168.2.6
7 0.002510 192.168.2.7
8 0.003115 192.168.2.7
9 0.003340 192.168.2.6
10 0.003408 192.168.2.6
11 0.003428 192.168.2.7
12 16.294894 192.168.2.7
13 16.295856 192.168.2.6
14 16.295921 192.168.2.6
15 16.295966 192.168.2.7
16 16.296641 192.168.2.7
17 16.296662 192.168.2.7
18 16.297045 192.168.2.6
19 16.297060 192.168.2.7
Frame 4: Packet, 365 bytes on wire (2920 bit
Linux cooked capture v2
Internet Protocol Version 4, Src: 192.168.2.
Transmission Control Protocol, Src Port: 449
Transport Layer Security
[Stream index: 0]
TLSv1.3 Record Layer: Handshake Protocol:
Content Type: Handshake (22)
Version: TLS 1.0 (0x0301)
Length: 288
Handshake Protocol: Client Hello
4 client pkts, 5 server pkts, 6 turns.
Entire conversation (4749 bytes)
Show as ASCII
No delta times
Stream 0
Find:
Case sensitive
Find Next
Help
Filter Out This Stream
Print
Save as...
Back
Close
Content Type (tls.record.content_type), 1 byte
Packets: 19 · Displayed: 19 (100.0%)
Profile: Default


capture_pfs.pcap
Wireshark · Follow TLS Stream (tls.stream eq 0) · capture_pfs.pcap
tls.stream eq 0
No. Time Source
4 0.000628 192.168.2.7
6 0.002504 192.168.2.6
8 0.003115 192.168.2.7
9 0.003340 192.168.2.6
10 0.003408 192.168.2.6
12 16.294894 192.168.2.7
13 16.295856 192.168.2.6
14 16.295921 192.168.2.6
16 16.296641 192.168.2.7
Frame 4: Packet, 365 bytes on wire (2920 bit
Linux cooked capture v2
Internet Protocol Version 4, Src: 192.168.2.
Transmission Control Protocol, Src Port: 449
Transport Layer Security
[Stream index: 0]
TLSv1.3 Record Layer: Handshake Protocol:
Content Type: Handshake (22)
Version: TLS 1.0 (0x0301)
Length: 288
Handshake Protocol: Client Hello
0 client pkts, 0 server pkts, 0 turns.
Entire conversation (0 bytes)
Show as ASCII
No delta times
Stream 0
Find:
Case sensitive
Find Next
Help
Filter Out This Stream
Print
Save as...
Back
Close
Transport Layer Security (tls), 293 bytes
Packets: 19 · Displayed: 9 (47.4%)
Profile: Default